



Computer Science Competition Region 2026 Programming Problem Set

I. General Notes

1. Do the problems in any order you like. They do not have to be done in order from 1 to 12.
2. All problems have a value of 60 points.
3. There is no extraneous input. All input is exactly as specified in the problem. Unless specified by the problem, integer inputs will not have leading zeros. Unless otherwise specified, your program should read to the end of file.
4. Your program should not print extraneous output. Follow the form exactly as given in the problem.
5. A penalty of 5 points will be assessed each time that an incorrect solution is submitted. This penalty will only be assessed if a solution is ultimately judged as correct.

II. Names of Problems

Number	Name
Problem 1	Avery
Problem 2	Dakota
Problem 3	Logan
Problem 4	Austin
Problem 5	Bingo
Problem 6	Bango
Problem 7	Bongo
Problem 8	Liz
Problem 9	Julia
Problem 10	Lune
Problem 11	Sciel
Problem 12	Renoir

1. Avery

Program Name: Avery.java

Input File: none

Avery is welcoming the UIL Robot Overlords with a portrait of their leader. Help spread the love to keep the robot army from taking drastic action.

Input:

None

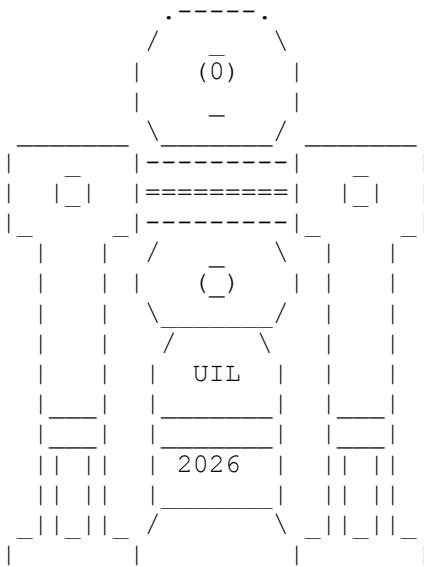
Output:

The robot shown below. The left-most characters have zero spaces at the beginning of the line. There are no additional spaces at the end of each line.

Sample input:

None

Sample output:



2. Dakota

Program Name: Dakota.java

Input File: dakota.dat

Dakota’s mom owns a parts warehouse. Dakota has been asked to help out by writing a program to read in a customer name and items from an order and then produce an invoice to provide to their customers. Their invoice should include calculations for subtotals and Texas Sales Tax at the rate of 8.25%.

Input:

The first line of the data file contains a single integer representing the number of orders to be processed.

For each order, the first line contains the customer name (as a single word). The next line contains an integer representing the number of different items having been ordered. Each of the next lines contain an item name (single word), quantity (an integer), and a unit price (a real number).

Output:

The invoice will contain section breaks which will consist of a single plus (+), 42 dashes (-), and end with a single plus (+). One of these such lines will also be the first and last line of each invoice. The first and last character of each of the remaining lines will be the “or” or “pipe” character (|), found above the return/enter key on your keyboard. There will be a single space as a “buffer” between the outer lines and the information inside. The title for the invoice is. “Invoice for C” where C is the customer name. In the next section, you will print the column titles spread out as follows: Item, followed by 9 spaces, Qty, 5 spaces, Price, 5 spaces, Subtotal. The next section will have each item starting under the I in Item above, the quantity right justified ending under the t of Qty above, the unit price with the decimals aligned under the i in Price above, and the item total with the decimals aligned under to o in Subtotal above. In the last section, we will have 3 lines: Subtotal, Tax, and Total. Each of those start with a one-space “buffer” before the words and each needs to be followed immediately by a :. Each of the values for these lines are aligned by the decimal which is aligned under the a in Subtotal in the title section up above. There will be no blank lines in between invoices—immediately start the next invoice.

Sample input:

```
2
BOB
1
Motor 3 38.79
RUMI
3
Servo 4 12.50
Wheel 2 18.75
Battery 1 49.99
```

Sample output:

```
+-----+
| Invoice for BOB |
+-----+
| Item          Qty      Price      Subtotal |
+-----+
| Motor          3        38.79      116.37 |
+-----+
| Subtotal:                116.37 |
| Tax:                      9.60 |
| Total:                   125.97 |
+-----+
+-----+
| Invoice for RUMI |
+-----+
| Item          Qty      Price      Subtotal |
+-----+
| Servo          4        12.50      50.00 |
| Wheel          2        18.75      37.50 |
| Battery        1        49.99      49.99 |
+-----+
| Subtotal:                137.49 |
| Tax:                      11.34 |
| Total:                   148.83 |
+-----+
```

3. Logan

Program Name: Logan.java

Input File: logan.dat

Logan's meat judging team is having a Cow Patty Bingo fundraiser. No one from the team wants to go around and actually locate the patties on the football field because no one wants to accidentally step on one. Logan attached a sensor to his drone that takes atmospheric readings and can "detect" the patties to a certain degree of certainty. The data is presented in a grid. Logan needs to display a "hazard map" and the list of one-based coordinate location(s) with the highest reading.

Input:

The first line contains an integer representing the number of test cases. For each test case the first line will contain 3 integers: R, C, and T representing the number of rows and columns in the grid and the threshold reading that indicates the likelihood of a cow patty being present in that grid location. This is followed by R rows, each with C readings.

Output:

Title each output case "--- Hazard Map Case X ---" where X is the test case number. Display the original grid with each reading right justified in a "cell" of 3 spaces with "Grid:" as a label at the beginning. Then a blank line. Then a line "Smelliest value: X" where X is the highest reading in the grid. Next a line "Location(s) :" followed by a list of all the 1-based coordinates where the highest reading happened. If there are more than one, simply separate them with a single space. This needs to be followed with a blank line. Finally, title the hazard grid "Hazard map:" and display the grid with . for "safe" zones and X for hazardous zones. These should be separated by a single space. Add a blank line to the end of each test case output.

Sample input:

```
1
3 5 70
55 70 61 88 40
72 68 70 59 90
33 71 45 70 70
```

Sample output:

```
--- Hazard Map Case 1 ---
Grid:
 55 70 61 88 40
 72 68 70 59 90
 33 71 45 70 70

Smelliest value: 90
Location(s) : (2,5)

Danger map:
. X . X .
X . X . X
. X . X X
```

4. Austin

Program Name: Austin.java

Input File: austin.dat

Your buddy Austin is a bit of a nuisance in class, and has been trying to pass you notes, but does not want the teacher to see them. He has started labelling each note with a sequence number, and passing them out of order, so she won't know what it is you're actually saying if one of the notes is intercepted. Once you have received all the message segments, put them in the correct order to read the message.

Input:

The input will begin with one integer, n ($0 < n \leq 1000$). Each test case will consist of one integer s ($1 \leq s \leq n$), denoting the sequence number, followed by a space, followed by the segment of the message, ($0 \leq \text{len}(\text{message_segment}) \leq 1000$), in brackets (the brackets are not part of the message). There will be no duplicate sequence numbers, and each message will only be made up of alphanumeric characters, spaces, and punctuation (`. , ? ! : ; ' " ( )`).

Output:

For each test case, output the message, after assembling it in the correct order.

Sample input:

```
4
3 [ good at mathema]
1 [This Teacher i]
4 [tics. LOL HAHABA]
2 [s not very]
```

Sample output:

```
This Teacher is not very good at mathematics. LOL HAHABA
```

5. Bingo

Program Name: Bingo.java

Input File: bingo.dat

While visiting your family for the holidays you decided to play bingo. While at the bingo hall your grandmother seemed to keep winning repeatedly. Of course, she credits it to her amazing bingo skill, but you suspect that her bingo card might be better than yours. To test this theory, you made note of your grandmother's bingo card and decided to simulate some games against other random bingo cards to see the results.

Input:

The first 5 lines of input will each contain 5 space separated numbers, each denoting a bingo slot on your grandmother's bingo card. The following line will contain a single integer N ($1 \leq N \leq 100$) denoting the number of test cases. Each game will start with a single integer C ($1 \leq C \leq 10$) denoting the number of bingo cards in this game (not including your grandmother's card) and a random seed to be used for the game. There will then be $5C$ following lines each with 5 space separated numbers, each 5 lines being a 5×5 bingo card.

To simulate a game, use the random seed provided and generate a value between 1 and 75 (inclusive) and mark the appropriate spaces on each bingo card. Once one or more players have filled a row, column, or diagonal they will win that game.

Note that the middle square of every card will contain 0, denoting a free space (already marked).

Output:

For each test case, simulate 1000 bingo games. Then, output what percentage of games your grandmother won in the format below.

Sample input:

```
12 30 40 50 74
13 24 35 60 63
6 27 0 47 69
1 22 45 58 66
8 26 33 46 72
1
2 523625323
7 22 38 51 64
3 18 42 47 71
12 29 0 58 68
1 25 33 60 62
15 17 45 53 75
5 20 40 49 66
11 27 35 55 73
2 16 0 60 70
14 30 43 46 63
9 23 31 58 69
```

Sample output:

```
33.5% Win Rate.
```

6. Bango

Program Name: Bango.java

Input File: bango.dat

You're interning at the popular game company Faucet for the summer, and they are running a couple of tournaments for their most popular game Diplomacy Squad 2. Your system has lots of performance statistics for each player, and they want you to produce the final leaderboard based on these statistics. The sorting is as follows:

First sort by Rank (highest – lowest), if rank is tied break the tie by win rate. If win rate is tied break that tie by combat score. If combat score is tied break that tie by headshot percentage. Finally, if after all this the two players are still tied, sort by username in lexicographical order.

Win Rate = wins / (wins + losses)

Combat Score = ((eliminations + assists * 0.5) / max(times_eliminated, 1))

Headshot Percentage = (headshots / max(eliminations, 1) * 100)

Input:

The first input line contains a single integer N ($1 \leq N \leq 1000$) denoting the number of players. Each of the following N lines will contain statistics for a single player in the following format - username, rank, wins, losses, eliminations, times_eliminated, assists, headshots. Rank is one of the following – Bronze, Silver, Gold, Platinum, Diamond.

Output:

Output the final leaderboard in the format below. Note there are 72 dashes in the top line.

Sample input:

```
6
ShadowByte Gold 8 2 142 30 55 80
NovaPulse Gold 8 2 142 30 55 80
IronFang Platinum 6 4 200 40 100 150
GhostWire Gold 8 2 142 30 55 92
ZeroKelvin Diamond 5 5 90 60 30 45
CipherX Platinum 6 4 200 40 100 160
```

Sample output:

PLACE	USERNAME	RANK	WIN%	COMBAT	HS%
#1	ZeroKelvin	[Diamond]	50.00	1.75	50.00
#2	CipherX	[Platinum]	60.00	6.25	80.00
#3	IronFang	[Platinum]	60.00	6.25	75.00
#4	GhostWire	[Gold]	80.00	5.65	64.79
#5	NovaPulse	[Gold]	80.00	5.65	56.34
#6	ShadowByte	[Gold]	80.00	5.65	56.34

7. Bongo

Program Name: Bongo.java

Input File: bongo.dat

Recently you've been getting way too many spam emails, enough that you've decided to do something about it. While reading about common classifier algorithms you come across an algorithm called Naïve Bayes and decide it would be perfect for this use case. The algorithm is called "naïve" because it assumes that all features (word in the email) are independent of each other, which simplifies the computation. Luckily there are some handy formulas to estimate these probabilities given below.

$$P(\text{Spam} \mid \text{words}) \propto P(\text{Spam}) \cdot \prod_i P(\text{word}_i \mid \text{Spam})$$

$$P(\text{Ham} \mid \text{words}) \propto P(\text{Ham}) \cdot \prod_i P(\text{word}_i \mid \text{Ham})$$

Also, to avoid zero probabilities for unseen words, you decide to use Laplace smoothing denoted below.

$$P(\text{word} \mid \text{class}) = \frac{\text{count}(\text{word}, \text{class}) + 1}{\text{total_words}(\text{class}) + V}$$

Where V is the size of the vocabulary (all words seen so far). Multiplying many small probabilities can quickly cause floating-point underflow. Instead of multiplying probabilities, you decide to sum log probabilities.

$$\log P(C \mid \text{words}) = \log P(C) + \sum_i \log P(\text{word}_i \mid C)$$

Note (the symbol $\cap$ represents the logical idea of "and" or intersection):

$$P(B \mid A) = \frac{P(A \cap B)}{P(A)}$$

Input:

The first input line is a single integer N ($1 \leq N \leq 10^6$) denoting the number of training emails. The next N lines will contain a label (spam/ham) and a string of space separated words making up the email. These inputs are used to train the model. The next input line is a single integer M ($1 \leq M \leq 10,000$) denoting the number of queries. The following M lines contain a string of space separated words making up an email.

Output:

For each email, output spam or ham based on the predicted class.

Sample input:

```
4 2
spam buy cheap products
ham meeting schedule tomorrow
spam special discount available
ham project meeting tomorrow
cheap discount
meeting tomorrow
```

Sample output:

```
spam
ham
```

8. Liz

Program Name: Liz.java **Input File:** liz.dat

You have recently become friends with twin sisters Liz and Julia, and they like to design puzzles for each other to solve, and have started including you in the fun. Liz wants to see if Julia can turn a given string into a palindrome by inserting characters, and wants you to determine the minimum number of inserted characters to turn the string into a palindrome.

Input:

The input will begin with one integer, n ($0 < n \leq 1000$). Each test case will consist of a string m , to be turned into a palindrome. Each string will be made up entirely of lowercase characters.

Output:

For each test case, output the minimum number of “insertions” required to turn the given string into a palindrome.

Sample input:

```
4
racecars
apples
bananas
tacocat
```

Sample output:

```
1
4
2
0
```

9. Julia

Program Name: Julia.java

Input File: julia.dat

Julia wants to get back at her sister with a better puzzle, as she got stumped by the palindrome “insertion” problem. She has decided to request your help in checking her work. She wants to see if, given a directed graph, are there any cycles in the graph, she does not think Liz knows anything about graphs. You will be given the number of nodes, and all the connections within the graph. A cycle is defined as a path in a graph that starts and ends at the same vertex, with no edges or other vertices repeated, forming a closed loop.

Input:

The input will begin with one integer, n ($0 < n \leq 1000$). Each test case will begin with 2 integers, m and c , denoting the number of nodes in the graph, and the number of connections you will be given, respectively. The following c lines will each contain a pair of 0-indexed integers (a b) denoting a connection from node index a to node index b , and do NOT indicate a connection from b to a exists as well.

Output:

For each test case, if there is a cycle in the graph, output the string “Lizn't that interesting.”, otherwise output “Juliaain't gonna beat me.”.

Sample input:

```
3
4 4
0 1
1 2
2 0
2 3
3 2
0 2
1 0
2 2
0 1
1 0
```

Sample output:

```
Lizn't that interesting.
Juliaain't gonna beat me.
Lizn't that interesting.
```

10. Lune

Program Name: Lune.java

Input File: lune.dat

Lune, a brilliant cryptographer, has discovered that XOR operations can reveal hidden patterns in encrypted messages. She has intercepted an array of encrypted values and needs to analyze various segments of the data by computing their XOR. More specifically, she needs to quickly answer many queries about the XOR value of different ranges in her intercepted data. You're not entirely sure why this would be useful to her, but every time you've asked her to clarify why this is important, she just says "when one falls, we continue." One way or another, she seems awfully certain that doing this is important, so you should probably help her.

Input:

The first line will consist of a single integer T , $1 \leq T \leq 100$, denoting the number of test cases to follow. Each of the following testcases will consist of at least three lines. The first line will consist of two single-space-separated integers n and q , $1 \leq n, q \leq 10^4$, denoting the number of encrypted values Lune intercepted, and the number of ranges that he is interested in querying. The second line will consist of n single-space-separated integers denoting the encrypted values he intercepted in the form of $A[0], A[1], \dots, A[n-1]$. The next q lines will each consist of two single-space-separated integers ℓ and r , denoting the starting index, and ending index, respectively, that Lune wishes to query over. For all q queries, $0 \leq \ell \leq r < n$ will hold. Also, you can assume that $\forall i, 0 \leq A[i] < 10^8$.

Output:

For each query, on its own line, print the value of $A[\ell] \oplus A[\ell+1] \oplus \dots \oplus A[r]$.

Sample input:

```
3
5 3
1 2 3 4 5
0 2
1 3
0 4
4 2
7 3 5 2
0 3
1 2
6 4
10 20 30 40 50 60
0 5
2 4
1 3
0 0
```

Sample output:

```
0
5
1
3
6
38
4
34
10
```

Explanation:

For the first test case, the first query resolves to $A[0] \oplus A[1] \oplus A[2] \Rightarrow 1 \oplus 2 \oplus 3 \Rightarrow 3 \oplus 3 = \boxed{0}$. The second query resolves to $A[1] \oplus A[2] \oplus A[3] \Rightarrow 2 \oplus 3 \oplus 4 \Rightarrow 1 \oplus 4 = \boxed{5}$.

For the second test case, the second query resolves to $A[1] \oplus A[2] \Rightarrow 3 \oplus 5 = \boxed{6}$.

For the third test case, the first query resolves to...

$$\begin{aligned}
 &A[0] \oplus A[1] \oplus A[2] \oplus A[3] \oplus A[4] \oplus A[5] \\
 &\Rightarrow 10 \oplus 20 \oplus 30 \oplus 40 \oplus 50 \oplus 60 \\
 &\Rightarrow 30 \oplus 30 \oplus 40 \oplus 50 \oplus 60 \\
 &\Rightarrow 0 \oplus 40 \oplus 50 \oplus 60 \\
 &\Rightarrow 40 \oplus 50 \oplus 60 \\
 &\Rightarrow 26 \oplus 60 = \boxed{38}
 \end{aligned}$$

11. Sciel

Program Name: Sciel.java

Input File: sciel.dat

Sciel is good friends with Sophie who recently got her addicted to a game called “Ticket to Ride”. In this game, each player obtains different train cards, which they can spend to complete their routes assigned to them. The board can effectively be abstracted out to a set of vertices (cities) and edges (a series of trains owned by at most a single player). A player is only allowed to place their trains along an edge between two cities so long as (1) there is an unclaimed edge between the two cities, and (2) they possess enough colored train cards corresponding to the number of colored slots for the unclaimed route. For non-adjacent cities, a player is said to connect them together if and only if there exists some set of contiguous edges between the two cities that the player owns. Given Sciel’s hand of colored train cards, the board, and a route consisting of a starting and ending city, help Sciel determine if she can complete her route by connecting the two cities together.

Input:

The first line will consist of a single integer T , $1 \leq T \leq 50$, denoting the number of testcases to follow. Each of the following T lines will consist of the serialized object string corresponding to the board, hand, and route that Sciel wishes to investigate. For each test case, you may assume that there are less than 100 cities, less than 500 edges, at most 15 of each colored card in Sciel’s hand, and colors consist of either Red, Green, Blue, Yellow, or Pink. You may also assume that each graph representation of the board will be undirected.

Output:

For each test case, on its own line, if it is possible to complete the route, then output the string “Tomorrow comes”, otherwise, instead print “When one falls, we continue”

Sample input: (lines truncated at ...)

2

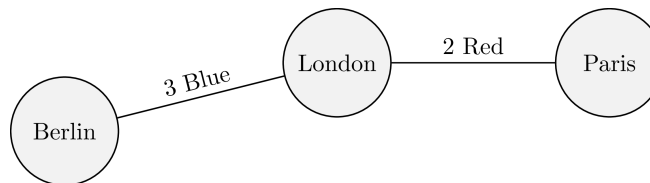
```
rO0ABXNyAAhUZsXN0Y2FzZQAAAAAAAAABAgADTAAFYm9hcmR0AAAdMQm9hcmQ7TAEEaGFuZHQABKxIYW5...
rO0ABXNyAAhUZsXN0Y2FzZQAAAAAAAAABAgADTAAFYm9hcmR0AAAdMQm9hcmQ7TAEEaGFuZHQABKxIYW5...
```

Sample output:

```
Tomorrow comes
When one falls, we continue
```

Explanation:

The first testcase consists of three cities (Berlin, London, and Paris), with an edge between Berlin and London costing three blue cards and an edge between London and Paris costing two red cards. Your hand consists of exactly two red cards and three blue cards. Your route is from Paris to Berlin. Since you have exactly enough cards to purchase the two required edges between the cities, it is possible to complete the route.



The second testcase is identical to the first but simply is missing enough of the blue cards to purchase the edge between Berlin and London. Since no other edges exist connecting the two cities, and there is no other way to get to Paris other than going through London, it is impossible to complete the route given her hand.

Important Notes:

There are many factors that affect how object serialization works that don’t pertain to what the object logically represents. Because of this, you should avoid changing the names of existing methods/fields, creating additional public/protected methods (private is fine) and avoid creating any new fields as this affects what data is associated with the encoding of the object. You may set the `toString()` method to simply return the empty string if you do not wish to make use of it in your implementation.

Starter Class:

```
import java.io.Serializable;
import java.util.HashMap;

class Route implements Serializable {
    private static final long serialVersionUID = 1L;

    protected String city1, city2;

    @Override
    public String toString() {
        // TODO: Implement
    }
}

enum Color {
    RED, GREEN, BLUE, YELLOW, PINK
}

class EdgeCost implements Serializable {
    private static final long serialVersionUID = 1L;

    protected Color color;
    protected int weight;

    @Override
    public String toString() {
        // TODO: Implement
    }
}

class Board implements Serializable {
    private static final long serialVersionUID = 1L;

    protected HashMap<String, HashMap<String, EdgeCost>> graph;

    @Override
    public String toString() {
        // TODO: Implement
    }
}

class Hand implements Serializable {
    private static final long serialVersionUID = 1L;

    protected HashMap<Color, Integer> cards;

    @Override
    public String toString() {
        return hand.toString();
    }
}

class Testcase implements Serializable {
    private static final long serialVersionUID = 1L;

    protected Board board;
    protected Hand hand;
    protected Route route;

    public String solve() {
        // TODO: Implement
    }

    @Override
    public String toString() {
        // TODO: Implement
    }
}
```

Serialization Code:

```
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.util.Base64;

class Serializer {
    public static String marshal(Serializable o) throws IOException {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(baos);
        oos.writeObject(o);
        String encodedData = Base64.getEncoder().encodeToString(baos.toByteArray());
        baos.close();
        oos.close();
        return encodedData;
    }

    public static Object unmarshal(String s) throws ClassNotFoundException, IOException {
        byte[] data = Base64.getDecoder().decode(s);
        ByteArrayInputStream bais = new ByteArrayInputStream(data);
        ObjectInputStream ois = new ObjectInputStream(bais);
        Object o = ois.readObject();
        bais.close();
        ois.close();
        return o;
    }
}
```

Sample Usage:

```
String byteStream = file.readLine();
Object o = null;
try {
    o = Serializer.unmarshal(byteStream);
} catch (Exception e) {
    System.out.println("Come on problem setter, get your classes straight!");
    return;
}

Testcase tc = null;
if (o instanceof Testcase) {
    tc = Testcase.class.cast(o);
} else {
    System.out.println("Come on problem setter, get your classes straight!");
    return;
}
```

12. Renoir

Program Name: Renoir.java

Input File: renoir.dat

Renoir is the manager at your local McNaldo's and has recently been put in charge of determining the different packaging count of McNuggets that they will offer for customers to buy. Renoir is a simple man, and plans to only offer 3 unique McNugget counts; however, he thought it would be funny to also make those three counts relatively prime as a set. Moreover, one of his employees suggested making one of the counts evenly divide the least common multiple of the other two, and so Renoir has decided to adopt this idea to help boost employee morale.

Interested in pleasing as many people as possible, Renoir wants to make sure that if a customer wanted to purchase an amount of McNuggets that they did not offer a specific count of, they would be able to achieve that count by purchasing some combination of the three options offered. Renoir is now interested in finding the largest count that is impossible to make via some combination of the three McNugget counts that he is considering to offer.

Input:

The first line will consist of a single integer T , $1 \leq T \leq 2.5 \cdot 10^5$, denoting the number of testcases to follow. Each of the following T lines will consist of three single-space-separated integers x , y , and z , $2 \leq x, y, z \leq 10^4$, denoting the three McNugget count sizes that Renoir is considering. It is guaranteed that x , y , and z are relatively prime as a set, and there exists only one of x , y , or z which evenly divides the least common multiple of the other two.

Output:

For each test case, on its own line, print the integer corresponding to the largest number of McNuggets that cannot be purchased given that customers can buy McNuggets in denominations of x , y , and z .

Sample input:

```
3
6 9 20
5 45 7
2 7 82
```

Sample output:

```
43
23
5
```

Explanation:

For the first test case, consider the following set of numbers and how they can/cannot be constructed given the three integers provided:

```
6 = (1) · 6
7 = ✗
8 = ✗
9 = (1) · 9
10 = ✗
11 = ✗
12 = (2) · 6
13 = ✗
```

```
14 = ✗
15 = (1) · 6 + (1) · 9
16 = ✗
17 = ✗
18 = (3) · 6
19 = ✗
20 = (1) · 20
21 = (2) · 6 + (1) · 9
```